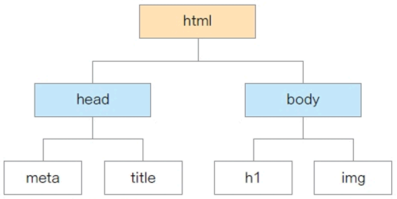
5. DOM(Document Of Model)

5-1 문서 객체 모델 알아보기

- **문서 객체 모델이란**
  - 자바스크립트를 이용하여 웹 문서에 접근하고 제어할 수 있도록 객체를 사용해 웹 문서를 체계적으로 정리하는 방법

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>DOM Tree 알아보기</title>
</head>
<body>
  <h1>Do it!</h1>
  <img src="images/doit.jpg" alt="공부하는 이미지">
</body>
</html>
```



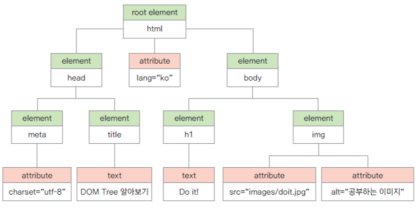
- **DOM 트리**
  - 웹 문서에 있는 요소들 간의 부모, 자식 관계를 계층 구조로 표시한 것
  - 나무 형태가 되기 때문에 "DOM 트리"라고 함.
  - 노드(node) : DOM 트리에서 가지가 갈라져 나간 항목
  - 루트 노트(root node) : DOM 트리의 시작 부분(html)

DOM 을 구성하는 원칙



- 모든 HTML 태그는 요소(element) 노드
- 웹 문서의 텍스트 내용은 요소 노드의 자식 노드인 텍스트(text) 노드
- 태그의 속성은 요소 노드의 자식 노드인 속성(attribute) 노드
- 주석은 주석(comment) 노드

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>DOM Tree 알아보기</title>
</head>
<body>
  <h1>Do it!</h1>
  <img src="images/doit.jpg" alt="공부하는 이미지">
</body>
</html>
```



5-2 DOM 요소에 접근하고 속성 가져오기

- **DOM 요소에 접근하기**
  - **getElement-* 함수 사용하기**

```
기본형  document.getElementById("id명")
        document.getElementsByClassName("class명")
        document.getElementsByTagName("태그명")
```

- **querySelector( ) 함수**

```
기본형  document.querySelector(선택자)
```

  - id 이름 앞에는 해시 기호(#), class 이름 앞에는 마침표(.), 태그는 기호 없이 태그명 사용
  - querySelecotorAll( ) 메서드는 반환 값이 여러 개일 때 모두 반환
    → 노드 리스트로 저장됨

▼ 예제 코드

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>DOM</title>
    <link rel="stylesheet" href="css/product.css">
</head>
<body>
    <div id="container">
        <h1 id="heading">에디오피아 게뎁</h1>
        <div id="prod-pic">
            <img src="images/coffee-pink.jpg" alt="에디오피아 게뎁" id="cup" width="200" he
ight="200">
                <div id="small-pic">
                    <img src="images/coffee-pink.jpg" class="small">
                    <img src="images/coffee-blue.jpg" class="small">
                    <img src="images/coffee-gray.jpg" class="small">
                </div>
        </div>
        <div id="desc">
            <ul>
                <li>상품명 : 에디오피아 게뎁</li>
                <li class="bluetext">판매가 : 9,000원</li>
                <li>배송비 : 3,000원<br>(50,000원 이상 구매시 무료)</li>
                <li>적립금 : 180원(2%)</li>
                <li>로스팅 : 2019.06.17</li>
                <button>장바구니 담기</button>
            </ul>
            <a href="#" id="view">상세 설명 보기</a>
        </div>

        <div id="detail">
            <hr>
            <h2>상품 상세 정보</h2>
            <ul>
                <li>원산지 : 에디오피아</li>
                <li>지 역 : 이르가체프 코체레</li>
                <li>농 장 : 게뎁</li>
                <li>고 도 : 1,950 ~ 2,000 m</li>
                <li>품 종 : 지역 토착종</li>
                <li>가공법 : 워시드</li>
            </ul>
            <h3>Information</h3>
            <p>2차 세계대전 이후 설립된 게뎁농장은 유기농 인증 농장으로 여성의 고용 창출과 지역사회 발전에
```

```
기여하며 3대째 이어져 내려오는 오랜 역사를 가진 농장입니다. 게뎁 농장은 SCAA 인증을 받은 커피 품질관리 실험
실을 갖추고 있어 철처한 관리를 통해 스페셜티커피를 생산합니다.</p>
            <h3>Flavor Note</h3>
            <p>은은하고 다채로운 꽃향, 망고, 다크 체리, 달달함이 입안 가득.</p>
        </div>
    </div>
</body>
</html>
```

이미지

- https://github.com/funnycom/doit-hcj-new/blob/main/19/results/images/coffee-blue.jpg
- https://github.com/funnycom/doit-hcj-new/blob/main/19/results/images/coffee-gray.jpg
- https://github.com/funnycom/doit-hcj-new/blob/main/19/results/images/coffee-pink.jpg

CSS

- https://github.com/funnycom/doit-hcj-new/blob/main/19/results/css/product.css

- **텍스트 내용에 접근**
  - innerText : 웹 브라우저 창에 보이는 텍스트 내용
  - innerHTML : 화면에 보이는 것과 상관없이 HTML 태그까지 포함한 텍스트 내용
  - textContent : 화면에 보이는 것과 상관없이 텍스트 내용

```
기본형  요소.innerText = 내용
        요소.innerHTML = 내용
        요소.textContent = 내용
```

```
기본형  이미지요소.src = 파일 경로
```

```
<div id="detail">
  <ul>
    <li>원산지 : 에디오피아</li>
    <li style="display: none">지 역 : 이르가체프 코체레</li>
    <li style="display: none">농 장 : 게뎁</li>
  </ul>
</div>

<script>
  let detail = document.querySelector('#detail > ul');
  console.log(detail.innerText);
  console.log(detail.innerHTML);
  console.log(detail.textContent);
</script>
```

- **DOM 요소에 함수 직접 연결하기**

```
<div class="container">
 <img src="images/cat.jpg" id="cat">
</div>

<script>
```

```
  let cat = document.querySelector("#cat");
  cat.onclick = () => alert("이미지를 클릭했습니다");
</script>
```

- **함수 이름을 사용해 연결하기**

```
<div class="container">
 <img src="images/kitty-1.png" id="cat">
</div>

<script>
  let cat = document.querySelector("#cat");
  cat.onclick = changePic; // cat을 클릭하면 changePic 함수 실행

  function changePic() {
    cat.src = "images/kitty-2.png";
  }
</script>
```

- 예제 코드

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        #container { width : 700px; margin: 0 auto; text-align:center;}
        #contents { border: 1px solid #555; width: 500px; height: 200px; text-align: ju
stify;}
        .input_box { width: 500px; }
    </style>
</head>
<body>
    <div id="container">
        <div id="contents"></div>
        <div class="input_box">
            <input type="text" name="txtContent" id="txtContent" size="40">
            <button id="insertBtn">추가하기....</button>
            <button onclick="clearContents();">리셋....</button>
        </div>
    </div>
    <script>
        function clearContents(){
            document.querySelector('#contents').innerHTML= '';
            document.querySelector('#txtContent').value ='';
        }

        let insertBtn = document.querySelector('#insertBtn');
        insertBtn.onclick= addContent;

        function addContent(){
            let txt = document.querySelector('#txtContent');
            let contents = document.querySelector('#contents');
```

```
            //contents.innerText += txt.value;
            contents.innerHTML += `${txt.value}<br>`;
            txt.value ='';
            console.log(txt.value);
        };
    </script>
</body>
</html>
```

5-3 DOM에서 이벤트 처리하기

- **DOM의 event 객체**
  - 웹 문서에서 이벤트 발생한 요소가 무엇인지,
    어떤 이벤트가 발생했는지 등의 정보가 담긴 객체

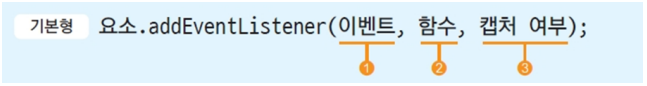
구분		설명
프로퍼티	altKey	이벤트가 발생할 때 Alt를 눌렀는지 여부를 boolean값으로 반환합니다.
	button	마우스에서 누른 버튼의 키값을 반환합니다.
	charCode	keypress 이벤트가 발생할 때 어떤 키를 눌렀는지 유니코드값으로 반환합니다.
	clientX	이벤트가 발생한 가로 위치를 반환합니다.
	clientY	이벤트가 발생한 세로 위치를 반환합니다.
	ctrlKey	이벤트가 발생했을 때 Ctrl를 눌렀는지 여부를 boolean값으로 반환합니다.
	pageX	현재 문서 기준으로 이벤트가 발생한 가로 위치를 반환합니다.
	pageY	현재 문서 기준으로 이벤트가 발생한 세로 위치를 반환합니다.
	screenX	현재 화면 기준으로 이벤트가 발생한 가로 위치를 반환합니다.
	screenY	현재 화면 기준으로 이벤트가 발생한 세로 위치를 반환합니다.
	shiftKey	이벤트가 발생할 때 Shift를 눌렀는지 여부를 boolean값으로 반환합니다.
	target	이벤트가 최초로 발생한 대상을 반환합니다.
	timeStamp	이벤트가 발생한 시간을 반환합니다.
	type	발생한 이벤트 이름을 반환합니다.
	which	키보드와 관련된 이벤트가 발생할 때 키의 유니코드값을 반환합니다.
메서드	preventDefault()	이벤트를 취소할 수 있는 경우에 취소합니다.

- **this 예약어 사용하기**
  - 이벤트가 발생한 대상에 접근할 때 사용하는 예약어
  - 화살표 함수에서는 사용할 수 없음

```
<div class="container">
  <img src="images/cat.jpg" id="cat">
</div>

<script>
  let cat = document.querySelector("#cat");
  cat.onclick = function(event) {
    alert(`이미지 파일: ${this.src}`);
  }
</script>
```

- **addEventListener( ) 함수 사용하기**
  - 이벤트 객체를 사용해 이벤트 처리기 연결



  - 예제 코드 >> **마우스 포인터를 올리면 이미지 바꾸기**

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM - addEventListener</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
    body {
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    img {
      padding: 20px;
      height: 250px;
      border: 1px solid #eee;
      border-radius: 10px;
      box-shadow: 2px 2px 5px #ccc;
    }
  </style>
</head>
<body>
  <div class="container">
    <img src="images/kitty-1.png" id="cat">
  </div>

    <script>
    // Do it! 이벤트 리스너로 이벤트 처리하기

    </script>
</body>
</html>
```

  - 이미지
    - https://github.com/funnycom/doit-hcj-new/blob/main/19/results/images/kitty-1.png
    - https://github.com/funnycom/doit-hcj-new/blob/main/19/results/images/kitty-2.png

- **CSS 속성에 접근하기**

기본형 document.요소명.style.속성명

👉 예) id가 desc인 요소의 글자를 파란색으로 변경하려면
document.querySelector("#desc").style.color = "blue";

```
<div id="container">
  <p>도형 위로 마우스 포인터를 올려놓으세요.</p>
  <div id="rect"></div>
</div>

<script>
  let rect = document.querySelector("#rect");
  rect.addEventListener("mouseover", () => {
    rect.style.backgroundColor = "green"; // rect 요소의 배경색
    rect.style.borderRadius = "50%"; // rect 요소의 테두리 둥글게 처리
  });
  rect.addEventListener("mouseout", () => {
    rect.style.backgroundColor = ""; // rect 요소의 배경색 지우기
    rect.style.borderRadius = ""; // rect 요소의 테두리 둥글게 처리하지 않음
  });
</script>
```

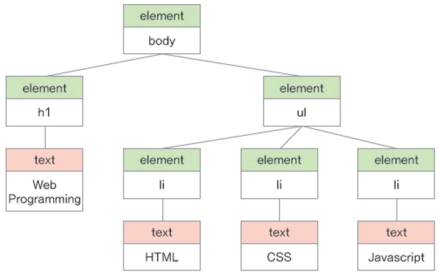
5-4 DOM에서 노드 추가, 삭제하기

- **노드 리스트란**
  - querySelectorAll( ) 메서드를 사용해 가져온 여러 개의 노드를 저장한 것

```
<h1>Web Programming</h1>
<ul id="itemList">
  <li>HTML</li>
  <li>CSS</li>
  <li>Javascript</li>
</ul>
```

👉 >document.querySelectorAll("li")

- **노드 추가**




```
<h1>Web Programming</h1>
<ul id="itemList">
  <li>HTML</li>
  <li>CSS</li>
  <li>Javascript</li>
</ul>
```

- **내용이 있는 텍스트 노드 추가**

> 1.**요소 노드 만들기 – createElement( ) 메서드**
> let newItem = document.***createElement***("li")
>
> **2. 텍스트 노드 만들기 – creatTextNode( ) 메서드**
> let textNode = document.***createTextNode***("Typescript")
>
> 3. **자식 노드 연결하기 – appendChild( ) 메서드**
> newItem.***appendChild***(textNode)
> document.querySelector('ul').***appendChild***(newItem)

- **속성 값이 있는 새로운 요소 추가**

```
<h1>Web Programming</h1>
<ul id="itemList">
  <li>HTML</li>
  <li>CSS</li>
  <li>Javascript</li>
</ul>
```

> 1. **요소 노드 만들기 – createElement( ) 메서드**
> **let newImg = document.createElement("img")**
>
> 2. **속성 추가하기**
> **newImg.src = "images/books.png"**
>
> 3. **자식 노드 연결하기**
> **document.body.appendChild(newImg)노ㄴㄴ**

- 노드 삭제
  - 노드 삭제

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Web Programming</h1>
    <ul id="lang">
        <li>Html</li>
        <li>Css</li>
        <li>JavaScript</li>
    </ul>
    <input type="text" name='txtNew' id="txtNew">
    <button id="btnInsert" >추가하기</button>
    <button id="deleteH1">제목삭제</button>
<script>

    document.querySelector('#deleteH1').addEventListener('click', ()=>{
        document.querySelector('h1').remove();
    });
    document.querySelector('#btnInsert').addEventListener('click', ()=>{
```

```
        let txtNew = document.querySelector('#txtNew');
        let newItem = document.createElement("li");
        let textNode = document.createTextNode(txtNew.value);
        let img = document.createElement('img');
        img.src = 'images/coffee-blue.jpg';

        newItem.appendChild(textNode);
        newItem.appendChild(img);
        document.querySelector('#lang').appendChild(newItem);
        txtNew.value = '';
    });



</script>
</body>
</html>
```

- 특정 리스트 삭제 하기..

```
<body>
    <h1>Web Programming</h1>
    <ul id="lang">
        <li>Html</li>
        <li>Css</li>
        <li>JavaScript</li>
    </ul>
<script>
const listItems = document.querySelectorAll('#lang li');
//console.log(listItems);
listItems.forEach(item =>{
    item.addEventListener("click", function(){
         //console.log(this);
        this.remove();
    });
});
</script>
</body>
```

- **클래스 속성 추가 삭제하기**

- LightBox
  - 이미지 다운로드
    https://github.com/kkh3363/kdt2025_cns_public/blob/main/%EA%B0%95%EC%9D%98%EC%9E%90%EB%A

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>라이트박스 만들기</title>
    <link rel="stylesheet" href="css/lightbox.css">
</head>
<body>
    <div class="container">
```

```
        <div class="row">
            <ul>
                <li> <img src="images/tree-1-thumb.jpg" data-src="images/tree-1.jpg" cl
ass="pic"></li>
                <li> <img src="images/tree-2-thumb.jpg" data-src="images/tree-2.jpg" cl
ass="pic"></li>
                <li> <img src="images/tree-3-thumb.jpg" data-src="images/tree-3.jpg" cl
ass="pic"></li>
                <li> <img src="images/tree-4-thumb.jpg" data-src="images/tree-4.jpg" cl
ass="pic"></li>
                <li> <img src="images/tree-5-thumb.jpg" data-src="images/tree-5.jpg" cl
ass="pic"></li>
                <li> <img src="images/tree-6-thumb.jpg" data-src="images/tree-6.jpg" cl
ass="pic"></li>
            </ul>
        </div>
        <div id="lightbox">
            <img src="images/tree-1.jpg" alt="" id="lightboxImage">
        </div>
    </div>
<script>
    let pics = document.querySelectorAll('.pic');
    let lightbox =document.querySelector('#lightbox');
    let lightboxImage =document.querySelector('#lightboxImage');

    for( let i=0;i< pics.length; i++){
        pics[i].addEventListener('click', showLightBox);
    }

    function showLightBox(){
        let bigLocation = this.dataset.src;
        lightboxImage.src =bigLocation;
        lightbox.style.display = 'block';
    }
    lightbox.onclick = ()=>{
        lightbox.style.display="none";
    };

</script>
</body>
</html>
```

- lightbox.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.row {
  width:420px;
  margin:0 auto;
```

```
}
.row ul {
  list-style:none;
  display: flex;
  flex-wrap: wrap;
}

/* 라이트 박스 스타일 */
#lightbox {
  position: fixed;  /* 위치 고정 */
  width:100%;  /* 너비 */
  height:100%;  /* 높이 */
  background-color:rgba(0,0,0,0.7);  /* 배경색 */
  top:0;  /* 시작 위치 - 위쪽 끝 */
  left:0; /* 시작 위치 - 왼쪽 끝 */
  display:none;  /* 화면에서 감추기 */
}

/* 라이트 박스 안의 이미지 */
#lightbox img {
  position:absolute;  /* top, left에 의해 위치 지정 */
  top:50%;  /* 위쪽에서 50% 부터 */
  left:50%;  /* 왼쪽에서 50% 부터 */
  transform:translate(-50%, -50%);  /* 요소를 화면 중앙에 표시하기 위해 이동 */
  border:5px solid #eee; /* 이미지 테두리 */
}
```

- Class 속성 추가 삭제

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>classList 추가/삭제 예제</title>
  <style>
    body {
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
    }
    div {
      margin-bottom: 20px;
    }
    button {
      padding: 10px 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
      cursor: pointer;
    }
    /* Do it! 필요한 클래스 스타일 정의하기 */
    .disabled {
      color: #ccc;
      background-color: #f0f0f0;
```

```
      border: 1px solid #ccc;
    }
    .enabled {
      color: #fff;
      background-color: #007bff;
      border: 1px solid #007bff;
    }
  </style>
</head>
<body>
  <div>
    <label>
      <input type="checkbox" id="agree"> 필수 항목 동의
    </label>
  </div>
  <div>
    <button id="proceed" disabled>다음으로 진행</button>
  </div>

  <script>
    // Do it! 체크 박스에 체크하면 버튼 활성화하기
    document.addEventListener('DOMContentLoaded', function() {
      const checkbox = document.querySelector('#agree');
      const proceedButton = document.querySelector('#proceed');

      checkbox.addEventListener('change', function() {
        if (this.checked) {  // 체크했다면
          proceedButton.classList.remove('disabled');  // .disabled 클래스 제거
          proceedButton.classList.add('enabled');  // .enabled 클래스 추가
          proceedButton.disabled = false;  // 버튼 활성화
        } else {  // 체크하지 않았다면
          proceedButton.classList.remove('enabled');  // .enabled 클래스 제거
          proceedButton.classList.add('disabled');  // .disabled 클래스 추가
          proceedButton.disabled = true;  // 버튼 비활성화
        }
      });
    });

    </script>
</body>
</html>
```

- button-2.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>classList toggle 함수 예제</title>
  <style>
    body {
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      height: 100vh;
    }
```

```
    div {
      margin-bottom: 20px;
    }
    button {
      padding: 10px 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
      cursor: pointer;
    }
    .disabled {
      color: #ccc;
      background-color: #f0f0f0;
      border: 1px solid #ccc;
    }
    .enabled {
      color: #fff;
      background-color: #007bff;
      border: 1px solid #007bff;
    }
  </style>
</head>
<body>
  <div>
    <label>
      <input type="checkbox" id="agree"> 필수 항목 동의
    </label>
  </div>
  <div>
    <button id="proceed" disabled>다음으로 진행</button>
  </div>

  <script>
    // Do it! 체크 박스에 체크하면 버튼 활성화하기 - toggle() 함수 사용
    document.addEventListener('DOMContentLoaded', function() {
      const checkbox = document.querySelector('#agree');
      const proceedButton = document.querySelector('#proceed');

      checkbox.addEventListener('change', function() {
        proceedButton.classList.toggle('enabled', this.checked);  // 체크했다면 .enabe
ld 토글
        proceedButton.classList.toggle('disabled', !this.checked);  // 체크하지 않았다면
.disabled 토글
        proceedButton.disabled = !this.checked;  // 버튼의 활성화/비활성화 상태
      });
    });
  </script>
</body>
</html>
```